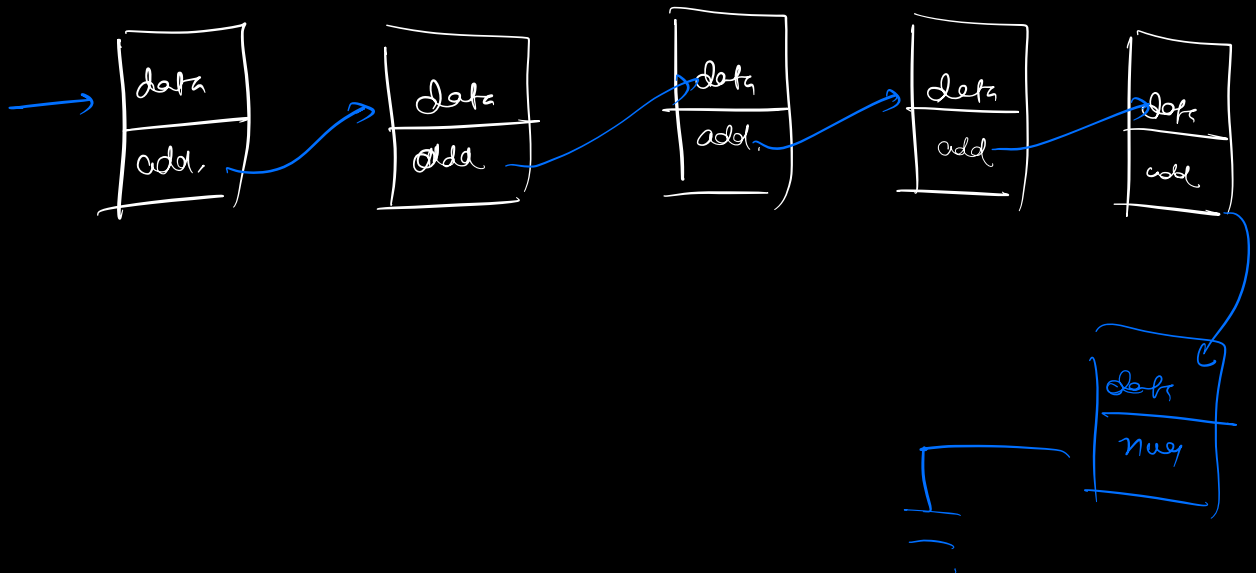
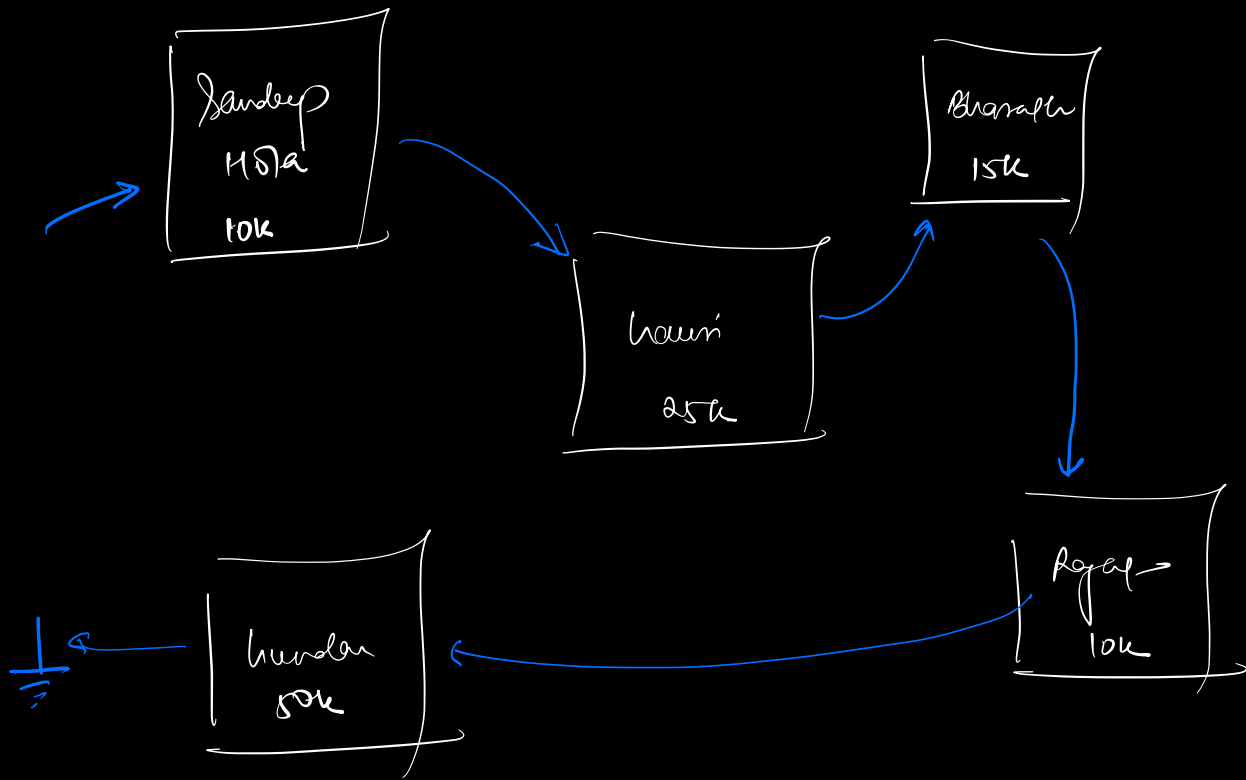




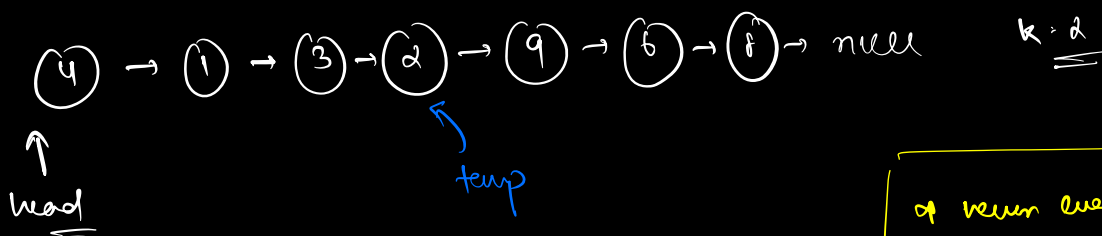
```

class Node {
    int data;
    Node next;

    Node (int u) {
        data = u;
        next = null;
    }
}

```

→ search an element in LL



```

boolean search(head, k) {

```

```

    if (head == null)
        return false;

```

```

    Node temp = head;

```

```

    while (temp != null) {

```

```

        if (temp.data == k)
            return true;

```

```

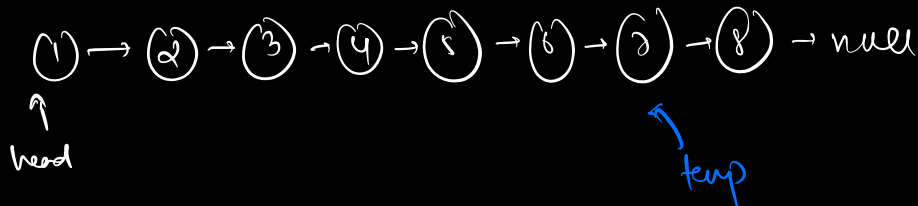
        temp = temp.next;
    }
    return false;
}

```

TC $\Rightarrow O(N)$

or never ever
change the
head

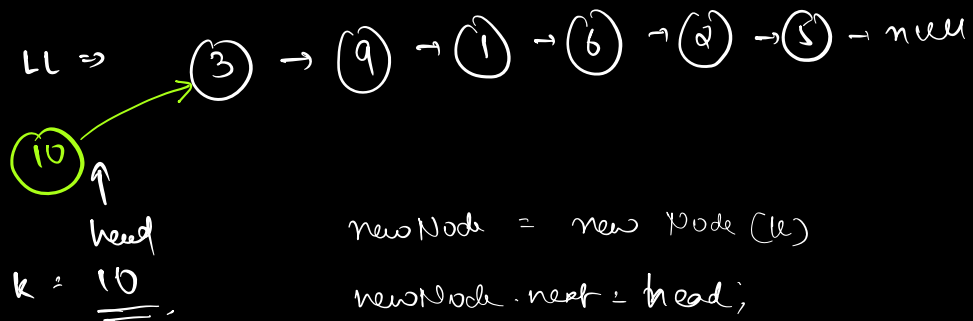
Sorted LL



k = 7

• Insertion in a LL:

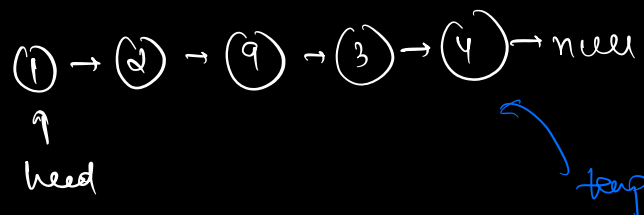
• At head:



```
newNode = new Node(k);
newNode.next = head;
head = newNode;
return head;
```

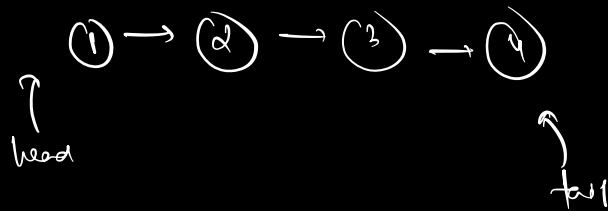
TC ⇒ O(1)

• at tail



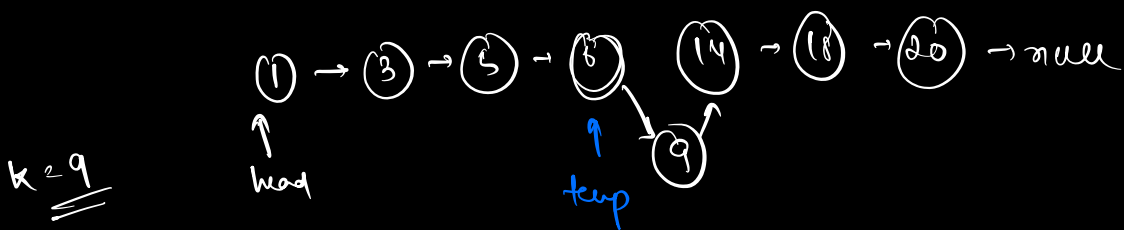
if (head == null) {
 }
 TC $\Rightarrow O(N)$

temp = head
 while (temp != null) {
 temp = temp.next
 }
 temp.next = new Node(k)



if we maintain a tail ptr. $\Rightarrow O(1)$

• insert a node in a sorted LL

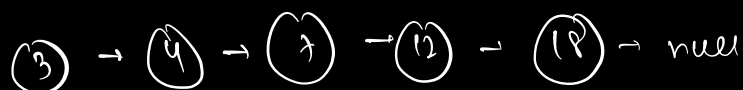


if (temp.data $\leq k$ & temp.next.data $> k$) {

tempNext = temp.next

temp.next = new Node(k)

newNode.next = tempNext



$k = 20$

if(temp.data <= k) {

if(temp.next == null ||

temp.next.data > k) {

tempNext = temp.next

temp.next = newNode(k)

newNode.next = tempNext

8

1 → 2 → 4 → 10 → null

↑
temp

}

k = 2

1 → 2 → 3 → 4 → 5 → null

↑
head

if(head.data > k)

insert k at head

• Deletion from a LL:

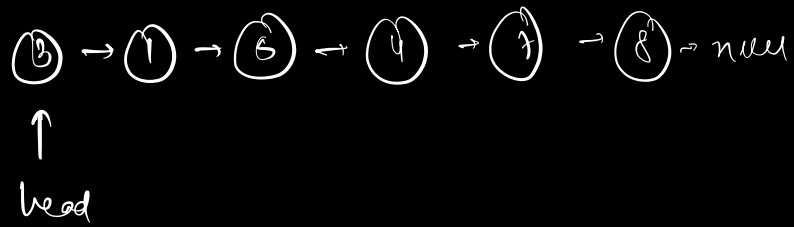
→ delet from head:

1 → 2 → 5 → 3 → 4 → null

↑
head

head = head.next;

• delete from tail:

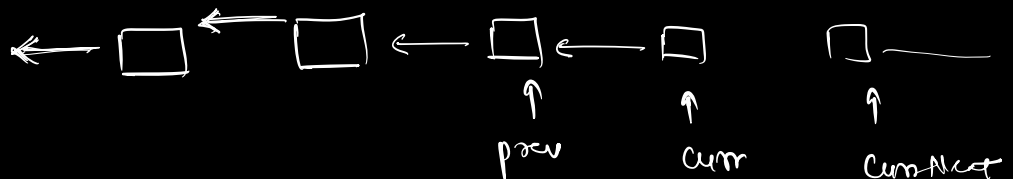
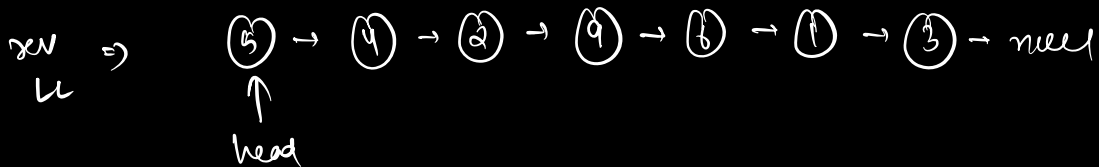
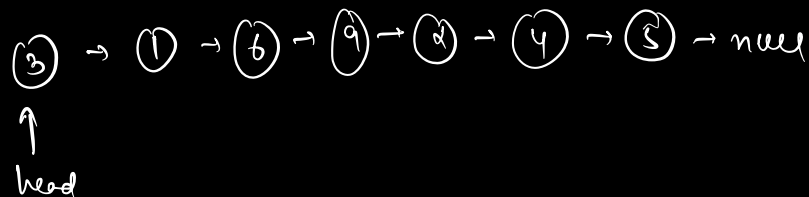


TC $\Rightarrow O(N)$

loop until $\rightarrow \text{temp.next.next} \neq \text{null}$

$\Rightarrow \text{temp.next} = \underline{\underline{\text{null}}}$

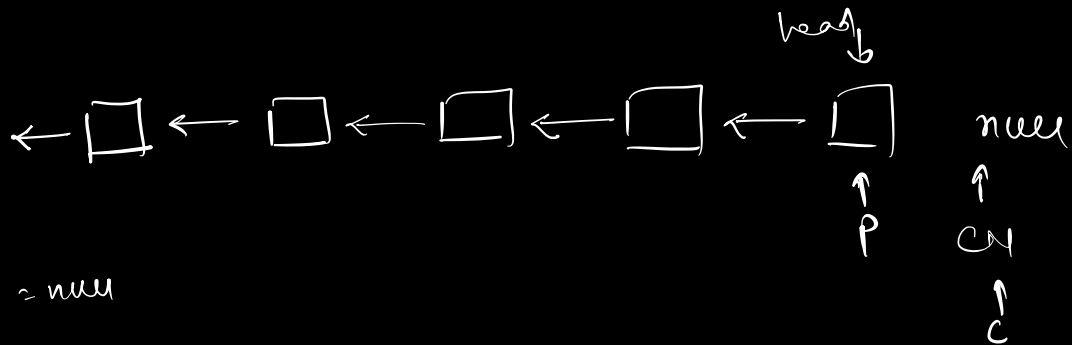
* reverse a LL:-



curr.next = prev
 prev = curr

curr = currNext

currNext = currNext.next



prev = null

while (c != null) {

c.next = p

p = c

c = c.next

if (c.next == null)

c.next = c.next

}

head = p

$T = O(N)$

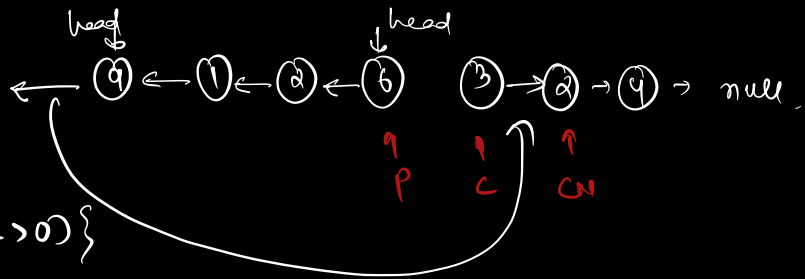
$S = O(1)$

⇒ Reverse the first k nodes of LL.

(9) → (1) → (2) → (6) → (3) → (2) → (4) → null

k = 4

prev = null



while (c != null && k > 0)

c.next = p

p = c

c = cn k--

if (cn != null)

cn = cn.next

}

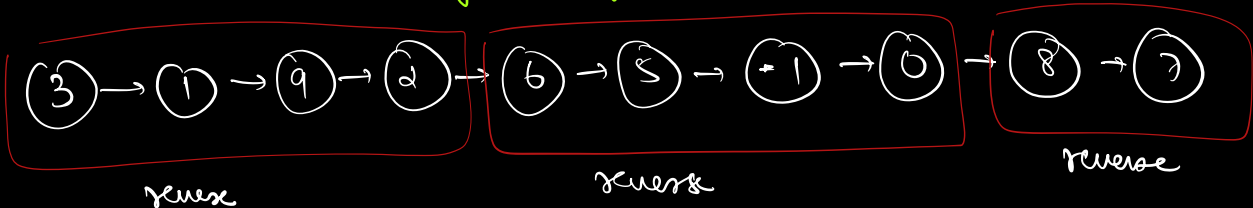
head, next = c

head = p

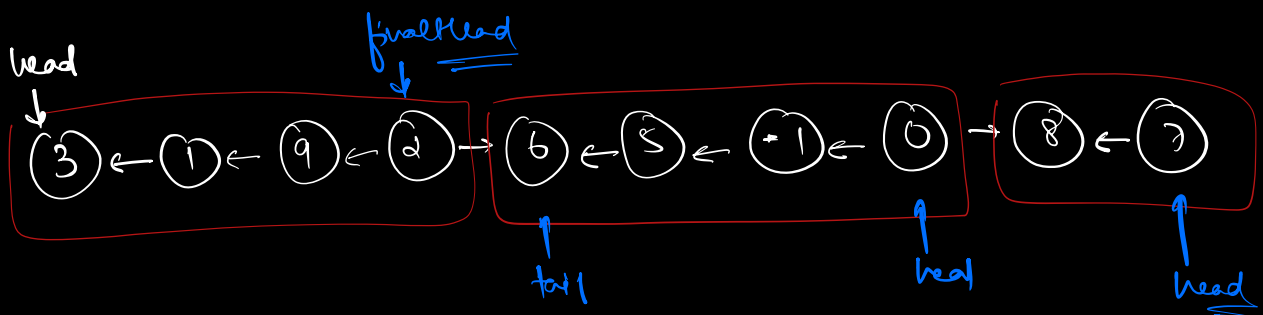
↓

6 → 2 → 1 → 9 → 3 → 2 → 4 → null

Reverse LL in groups of k size k=4



O/p = (2) → (9) → (1) → (3) → (0) → (4) → (5) → (6) → (7) → (8)



$$\left[\text{H}^+ \omega \rightarrow \underline{\underline{\delta Q^m}} \right]$$